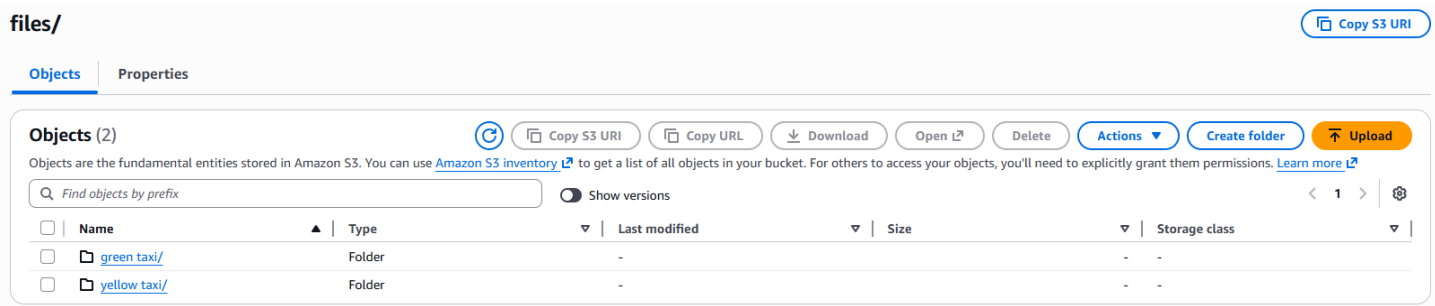
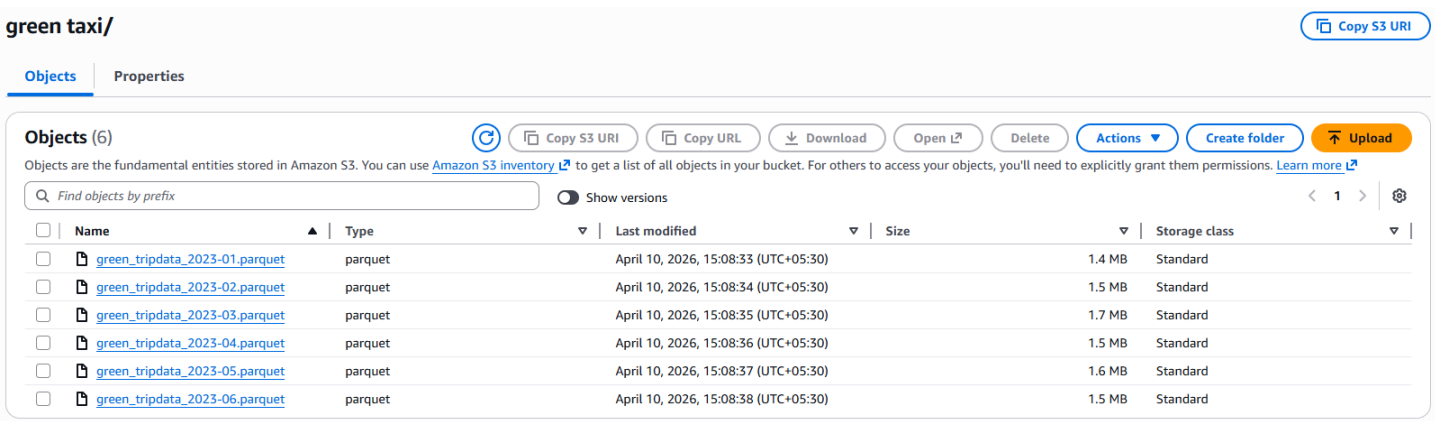
Name : Mihir Mungara

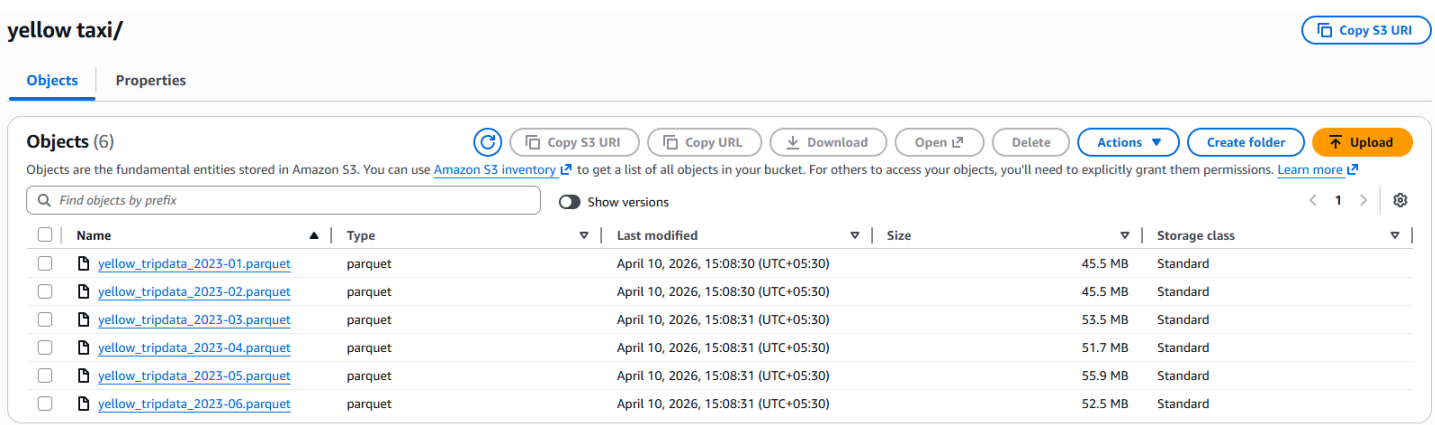
Redshift Assignment

Task 1: Redshift Spectrum

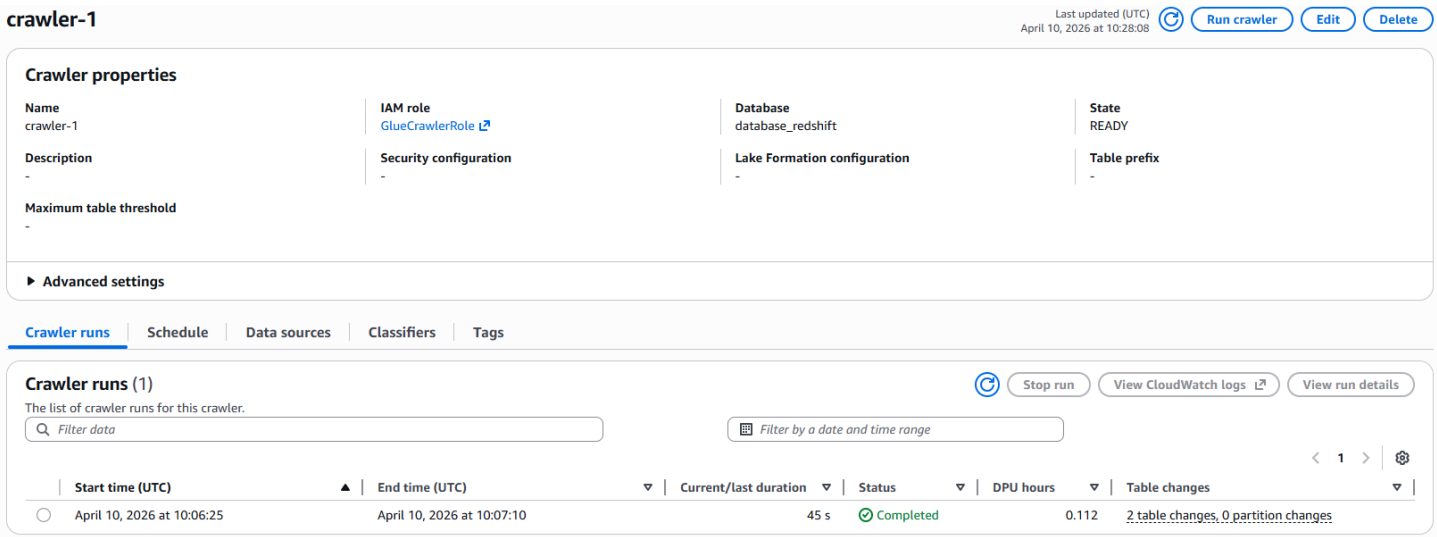
- Upload the required Yellow and Green Taxi Parquet files to Amazon S3.

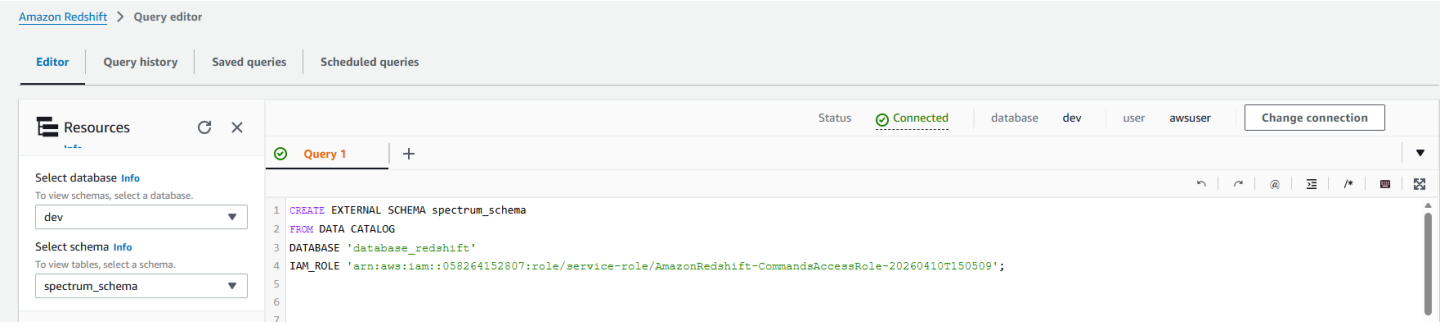


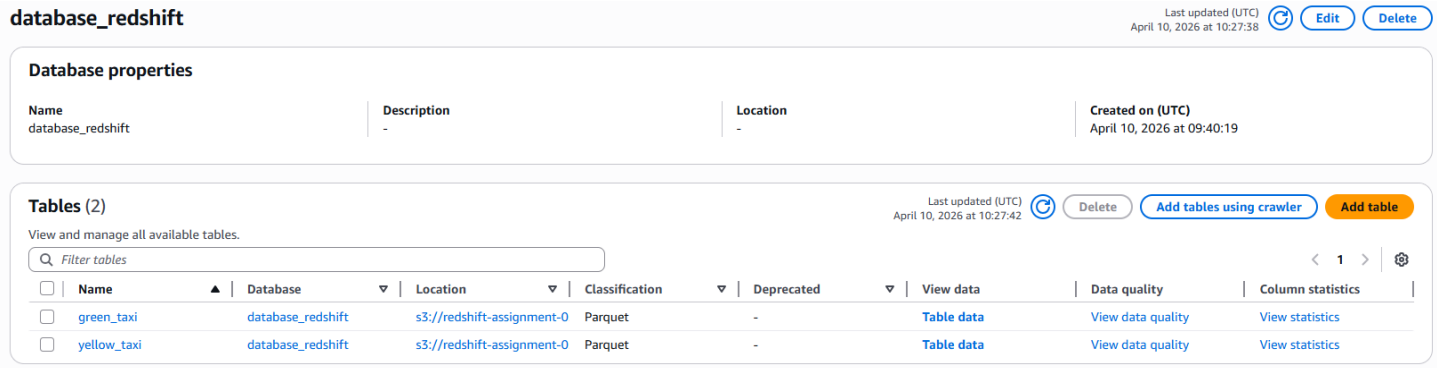




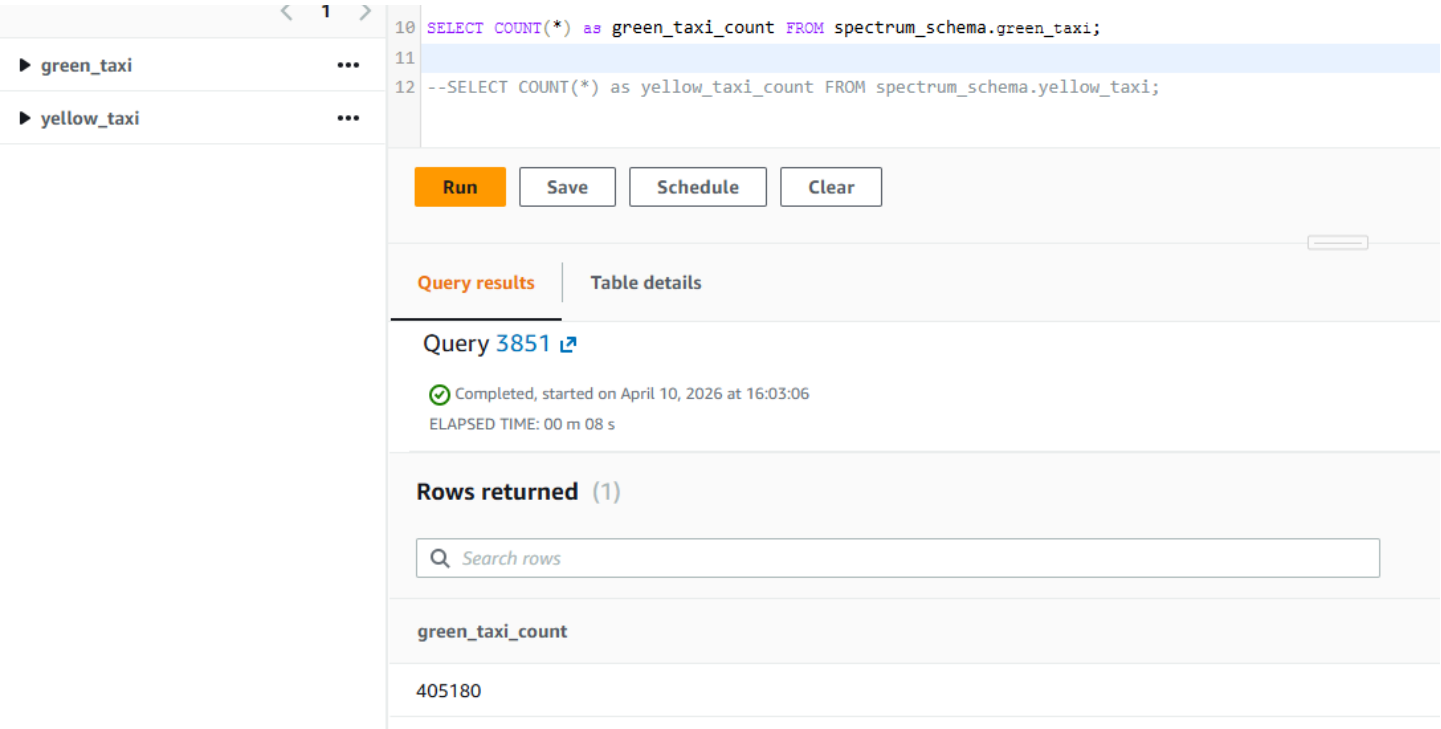
- Create an external schema using AWS Glue Data Catalog.
- Define external tables for both Yellow and Green Taxi data.

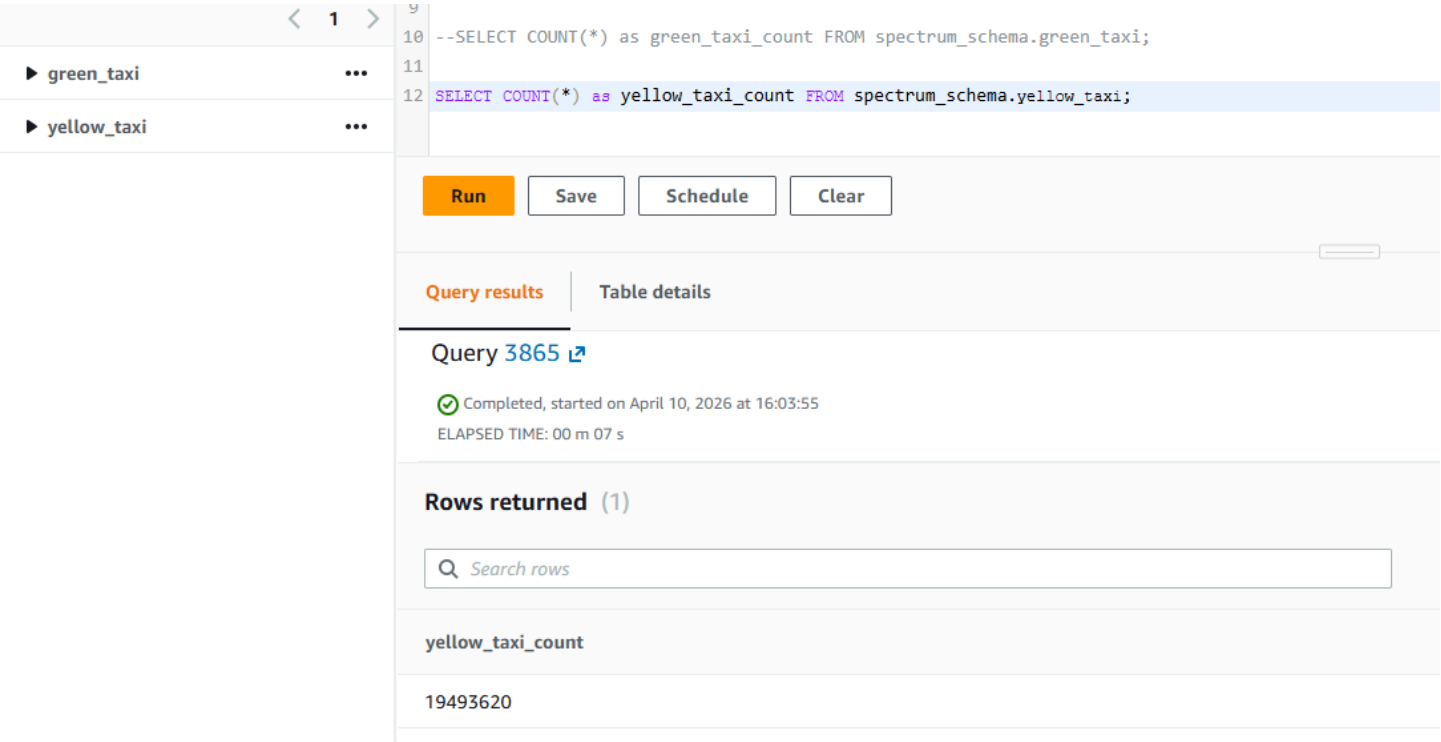






- Run analytical queries directly on the external tables to understand how Redshift Spectrum works.





Task 2: Analytical Queries

- Find the total number of trips per day, broken down by payment type and taxi type (Green or Yellow).

```
with yellow_pdt as (
  select DATE(tpep_pickup_datetime) as pickup_date,
         payment_type,
         'yellow' as taxi_type,
         count(*) as total_trips_per_day
  from spectrum_schema.yellow_taxi
  group by pickup_date, payment_type
  order by pickup_date, payment_type
),
green_pdt as (
  select DATE(lpep_pickup_datetime) as pickup_date,
         cast(payment_type as BIGINT),
         'green' as taxi_type,
         count(*) as total_trips_per_day
  from spectrum_schema.green_taxi
  group by pickup_date, payment_type
  order by pickup_date, payment_type
)
select * from yellow_pdt
union all
select * from green_pdt;
```

Rows returned (1852)

Export

Search rows

1 2 3 4 5 6 7 ... 186

pickup_date	payment_type	taxi_type	total_trips_per_day
2023-07-11	2	yellow	18207
2023-07-08	2	yellow	17310
2023-07-26	4	yellow	1427
2023-07-05	0	yellow	1947
2023-07-17	4	yellow	1301
2023-09-03	1	yellow	57819
2023-11-28	3	green	7
2023-11-20	3	green	10
2023-11-23	1	green	888
2023-11-29	1	green	1544
2023-11-19	3	green	6
2023-11-26	3	green	17
2023-11-14	1	green	1503
2023-11-09	3	green	16

- Find the top 10 trips by fare amount, showing:
    - Taxi type
    - Pickup location zone
    - Drop-off location zone

```sql
with yellow_t as(
  select 'yellow' as taxi_type,
         PULocationID as pickup_location_zone,
         DOLocationID as dropoff_location_zone,
         fare_amount
from spectrum_schema.yellow_taxi
),
green_t as(
  select 'green' as taxi_type,
         PULocationID as pickup_location_zone,
         DOLocationID as dropoff_location_zone,
         fare_amount
from spectrum_schema.green_taxi
),
combined as(
select * from green_t
union all
select * from yellow_t
)
select * from combined
order by fare_amount desc
limit 10;
```

Query 5398

Execution | Data | Visualize

Completed, started on April 10, 2026 at 18:20:21
ELAPSED TIME: 00 m 07 s

Rows returned (10)

Export

Search rows

1

taxi_type	pickup_location_zone	dropoff_location_zone	fare_amount
yellow	239	132	187502.96
yellow	249	90	143163.45
yellow	43	264	19152.9
yellow	163	264	12015.47
yellow	48	125	6339
green	92	265	3523.3
yellow	233	40	2320.11
yellow	145	145	2194.7
yellow	132	265	1916.8
yellow	132	265	1726.6

- Find the average trip distance for trips with more than 2 passengers, broken down by taxi type (Green or Yellow).

```sql
with yellow_t as(
select round(avg(trip_distance), 2) as average_trip_distance
from spectrum_schema.yellow_taxi
where passenger_count > 2
),
green_t as(
select round(avg(trip_distance), 2) as average_trip_distance
from spectrum_schema.green_taxi
where passenger_count > 2
)

select 'yellow' as taxi_type,
       average_trip_distance
from yellow_t

union all

select 'green' as taxi_type,
       average_trip_distance
from green_t
```

Query results | Table details

Query 5516

Completed, started on April 10, 2026 at 18:31:09

ELAPSED TIME: 00 m 07 s

Rows returned (2)

Search rows

taxi_type	average_trip_distance
yellow	3.9200000000000004
green	3.1399999999999997

Task 3: Load Data into Amazon Redshift

- Create internal tables on Amazon Redshift for Yellow and Green Taxi data.

```sql
CREATE TABLE public.green_taxi (
    vendorid INTEGER,
    lpep_pickup_datetime TIMESTAMP,
    lpep_dropoff_datetime TIMESTAMP,
    store_and_fwd_flag VARCHAR,
    ratecodeid BIGINT,
    pulocationid INTEGER,
    dolocationid INTEGER,
    passenger_count BIGINT,
    trip_distance FLOAT,
    fare_amount FLOAT,
    extra FLOAT,
    mta_tax FLOAT,
    tip_amount FLOAT,
    tolls_amount FLOAT,
    ehail_fee FLOAT,
    improvement_surcharge FLOAT,
    total_amount FLOAT,
    payment_type BIGINT,
    trip_type BIGINT,
    congestion_surcharge FLOAT
);
```

```sql
CREATE TABLE public.yellow_taxi (
    vendorid INTEGER,
    tpep_pickup_datetime TIMESTAMP,
    tpep_dropoff_datetime TIMESTAMP,
    passenger_count BIGINT,
    trip_distance FLOAT,
    ratecodeid BIGINT,
    store_and_fwd_flag VARCHAR,
    pulocationid INTEGER,
    dolocationid INTEGER,
    payment_type BIGINT,
    fare_amount FLOAT,
    extra FLOAT,
    mta_tax FLOAT,
    tip_amount FLOAT,
    tolls_amount FLOAT,
    improvement_surcharge FLOAT,
    total_amount FLOAT,
    congestion_surcharge FLOAT,
    airport_fee FLOAT
);
```

- Load the data from Amazon S3 using the COPY command.
- Initially, create the tables without defining any distribution key or sort key.

```sql
copy public.yellow_taxi
from 's3://redshift-assignment-1/yellow taxi/'
iam_role 'arn:aws:iam::381492274790:role/service-role/AmazonRedshift-CommandsAccessRole-20260413T095143'
format as parquet;
```

```sql
copy public.green_taxi
from 's3://redshift-assignment-1/green taxi/'
iam_role 'arn:aws:iam::381492274790:role/service-role/AmazonRedshift-CommandsAccessRole-20260413T095143'
format as parquet;
```

Task 4: Federated Query

1. Create a small lookup table (for example, payment type details or location metadata) in an external PostgreSQL database (Amazon RDS).

```
~ $ psql -h redshift-assignment-database.cjmemcak6h51.us-east-1.rds.amazonaws.com -U postgres
Password for user postgres:
psql (15.16, server 17.6)
WARNING: psql major version 15, server major version 17.
         Some psql features might not work.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

postgres=> CREATE TABLE payment_type_lookup (
postgres(>     payment_type INT PRIMARY KEY,
postgres(>     payment_description VARCHAR(50)
postgres(> );
CREATE TABLE
postgres=>
postgres=> INSERT INTO payment_type_lookup VALUES
postgres-> (1, 'Credit Card'),
postgres-> (2, 'Cash'),
postgres-> (3, 'No Charge'),
postgres-> (4, 'Dispute'),
postgres-> (5, 'Unknown'),
postgres-> (6, 'Voided Trip');
INSERT 0 6
```

2. Use Redshift Federated Query to join the taxi trip data in Redshift with the external PostgreSQL table at query time.

```sql
SELECT
    DATE(t.tpep_pickup_datetime) AS trip_date,
    p.payment_description,
    COUNT(*) AS total_trips
FROM public.yellow_taxi t
JOIN rds_schema.payment_type_lookup p
ON t.payment_type = p.payment_type
GROUP BY trip_date, p.payment_description
ORDER BY trip_date;
```

Rows returned (746)

Export

Search rows

1 2 3 4 5 6 7 ... 75

trip_date	payment_description	total_trips
2002-12-31	Credit Card	3
2002-12-31	Cash	1
2008-12-31	Cash	2
2008-12-31	Credit Card	6
2009-01-01	Cash	6
2009-01-01	Credit Card	5
2009-01-01	No Charge	2
2023-06-30	Credit Card	36
2023-06-30	Cash	4
2023-06-30	Dispute	1

```sql
SELECT
    DATE(t.lpep_pickup_datetime) AS trip_date,
    p.payment_description,
    COUNT(*) AS total_trips
FROM public.green_taxi t
JOIN rds_schema.payment_type_lookup p
ON t.payment_type = p.payment_type
GROUP BY trip_date, p.payment_description;
```

Rows returned (738)

Export

Search rows

1 2 3 4 5 6 7 ... 74

trip_date	payment_description	total_trips
2023-07-01	Credit Card	1034
2023-07-01	Cash	638
2023-07-02	No Charge	19
2023-07-02	Dispute	6
2023-07-03	Credit Card	977
2023-07-04	Cash	359
2023-07-05	Credit Card	1269
2023-07-05	No Charge	13
2023-07-06	Cash	647
2023-07-06	Credit Card	1286

3. Execute queries that retrieve descriptive information from the external database without ingesting it into Redshift.

--> Done in Above Images

Task 5: Baseline Performance Analysis

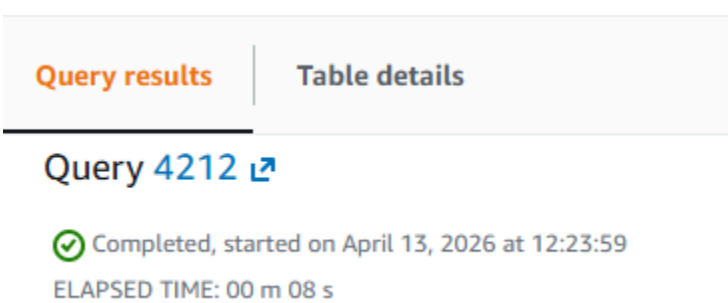
- Execute the analytical queries from Task 2 on the unoptimized Redshift tables.
- Capture and record:
  - Query execution time
  - Query execution plan
  - Data scanned and data redistribution metrics

Query 1 :-

```
with yellow_pdt as (
    select DATE(tpep_pickup_datetime) as pickup_date,
           payment_type,
           'yellow' as taxi_type,
           count(*) as total_trips_per_day
    from spectrum_schema.yellow_taxi
    group by pickup_date, payment_type
    order by pickup_date, payment_type
),
green_pdt as (
    select DATE(lpep_pickup_datetime) as pickup_date,
           cast(payment_type as BIGINT),
           'green' as taxi_type,
           count(*) as total_trips_per_day
    from spectrum_schema.green_taxi
    group by pickup_date, payment_type
    order by pickup_date, payment_type
)

select * from yellow_pdt
union all
select * from green_pdt;
```

Execution Time :-



Execution Plan :-

QUERYPLAN
XN Append (cost=200700200.00..401401400.00 rows=80000 width=16)
-> XN Subquery Scan "*SELECT* 1" (cost=200700200.00..200700700.00 rows=40000 width=16)
-> XN HashAggregate (cost=200700200.00..200700300.00 rows=40000 width=16)
-> XN S3 Query Scan yellow_taxi (cost=200000000.00..200400200.00 rows=40000000 width=16)
-> S3 HashAggregate (cost=200000000.00..200000200.00 rows=40000000 width=16)
-> S3 Seq Scan spectrum_schema.yellow_taxi location:"s3://redshift-assignment-1/yellow taxi/" format:PARQUET (cost=0.00..125000000.00 rows=10000000000 width=16)
-> XN Subquery Scan "*SELECT* 2" (cost=200700200.00..200700700.00 rows=40000 width=16)
-> XN HashAggregate (cost=200700200.00..200700300.00 rows=40000 width=16)
-> XN S3 Query Scan green_taxi (cost=200000000.00..200400200.00 rows=40000000 width=16)
-> S3 HashAggregate (cost=200000000.00..200000200.00 rows=40000000 width=16)
-> S3 Seq Scan spectrum_schema.green_taxi location:"s3://redshift-assignment-1/green taxi/" format:PARQUET (cost=0.00..125000000.00 rows=10000000000 width=16)

Data Scanned :-

```
select query,
       sum(bytes) / 1024 as mb_scanned,
       sum(rows) as rows_scanned
from stl_scan
where query = 4893
group by query;
```

query	mb_scanned	rows_scanned
4893	225	7479

data redistribution metrics :-

```
select query,
       step,
       label,
       rows,
       bytes,
       is_diskbased
from svl_query_summary
where query = 4893
order by step;
```

query	step	label	rows	bytes	is_diskbased
4893	0	scan tbl=92244314 name=S3 Subquery dev_spectrum_schema_yellow_taxi	971	19420	f
4893	0	scan tbl=4796 name=Internal Worktable	1852	74080	f
4893	0	scan tbl=4793 name=Internal Worktable	930	22320	f
4893	0	scan tbl=4795 name=Internal Worktable	922	22128	f
4893	0	scan tbl=30026574 name=S3 Subquery dev_spectrum_schema_green_taxi	952	19040	f
4893	0	scan tbl=1698765702 name=Internal Worktable	1852	74080	f
4893	1	project	971	0	f
4893	1	project	952	0	f
4893	1	return	1852	54638	f
4893	1	project	1852	0	f
4893	1	aggr tbl=443797718	930	29760	f
4893	1	aggr tbl=1448066811	922	29504	f
4893	2	project	971	0	f
4893	2	project	922	0	f
4893	2	project	930	0	f
4893	2	project	952	0	f
4893	2	return	1852	3040	f
4893	3	project	930	0	f
4893	3	project	922	0	f
4893	3	aggr tbl=216203559	922	29504	f
4893	3	aggr tbl=242390604	930	29760	f
4893	4	dist	930	22320	f
4893	4	dist	922	22128	f
4893	5	save tbl=1698765702	930	37200	f
4893	5	save tbl=1698765702	922	36880	f
4893	6	merge	0	0	f
4893	6	merge	0	0	f
4893	7	aggr tbl=2024851221	0	0	f
4893	7	aggr tbl=2037538601	0	0	f
4893	8	project	0	0	f
4893	8	project	0	0	f

Query 2 :-

```sql
with yellow_t as(
    select 'yellow' as taxi_type,
           PULocationID as pickup_location_zone,
           DOLocationID as dropoff_location_zone,
           fare_amount
    from spectrum_schema.yellow_taxi
),
green_t as(
    select 'green' as taxi_type,
           PULocationID as pickup_location_zone,
           DOLocationID as dropoff_location_zone,
           fare_amount
    from spectrum_schema.green_taxi
),
combined as(
    select * from green_t
    union all
    select * from yellow_t
)

select * from combined
order by fare_amount desc
limit 10;
```

Execution Time :-

Query 5048

Completed, started on April 13, 2026 at 13:41:29

ELAPSED TIME: 00 m 07 s

Execution Plan :-

```
QUERYPLAN
XN Subquery Scan combined  (cost=3003721928095.57..3003721928095.69 rows=10 width=48)
"  ->  XN Limit  (cost=3003721928095.57..3003721928095.59 rows=10 width=16)"
"        ->  XN Merge  (cost=3003721928095.57..3003721928095.62 rows=20 width=16)"
"              Merge Key: fare_amount"
"              ->  XN Network  (cost=3003721928095.57..3003721928095.62 rows=20 width=16)"
"                    Send to leader"
"                    ->  XN Sort  (cost=3003721928095.57..3003721928095.62 rows=20 width=16)"
"                          Sort Key: fare_amount"
"                          ->  XN Append  (cost=1001860964047.44..2003721928095.14 rows=20 width=16)"
"                                ->  XN Subquery Scan ""*SELECT* 1""  (cost=1001860964047.44..1001860964047.57 rows=10 width=16)"
"                                      ->  XN Limit  (cost=1001860964047.44..1001860964047.47 rows=10 width=16)"
"                                            ->  XN Sort  (cost=1001860964047.44..1001885964047.44 rows=10000000000 width=16)"
"                                                  Sort Key: fare_amount"
"                                                  ->  XN S3 Query Scan green_taxi  (cost=0.00..200000000.00 rows=10000000000 width=16)"
"                                                        ->  S3 Seq Scan spectrum_schema.green_taxi location:""s3://redshift-assignment-1/green taxi/"" format:PARQUET  (cost=0.00..100000000.00 rows=10000000000 width=1
"                          ->  XN Subquery Scan ""*SELECT* 2""  (cost=1001860964047.44..1001860964047.57 rows=10 width=16)"
"                                      ->  XN Limit  (cost=1001860964047.44..1001860964047.47 rows=10 width=16)"
"                                            ->  XN Sort  (cost=1001860964047.44..1001885964047.44 rows=10000000000 width=16)"
"                                                  Sort Key: fare_amount"
"                                                  ->  XN S3 Query Scan yellow_taxi  (cost=0.00..200000000.00 rows=10000000000 width=16)"
"                                                        ->  S3 Seq Scan spectrum_schema.yellow_taxi location:""s3://redshift-assignment-1/yellow taxi/"" format:PARQUET  (cost=0.00..100000000.00 rows=10000000000 width
```

Data Scanned :-

query	mb_scanned	rows_scanned
5048	292	19198586

Data redistribution metrics :-

query	step	label	rows	bytes	is_diskbased
5048	0	scan tbl=1015223380 name=Internal Worktable	20	720	f
5048	0	scan tbl=49979117 name=S3 Scan dev_spectrum_schema_green_taxi	381880	6110080	f
5048	0	scan tbl=1553038933 name=S3 Scan dev_spectrum_schema_yellow_taxi	18816606	301065696	f
5048	0	scan tbl=700808144 name=Internal Worktable	20	720	f
5048	0	scan tbl=1811307026 name=Internal Worktable	20	720	f
5048	0	scan tbl=1053058945 name=Internal Worktable	40	1440	f
5048	0	merge	0	0	f
5048	1	sort tbl=1174481561	20	0	f
5048	1	limit	11	0	f
5048	1	project	18816606	0	f
5048	1	project	381880	0	f
5048	1	project	20	0	f
5048	1	project	20	0	f
5048	1	return	20	720	f
5048	2	save tbl=1811307026	20	720	f
5048	2	sort tbl=1638620262	20	0	f
5048	2	sort tbl=1133493139	20	0	f
5048	2	project	10	0	f
5048	3	save tbl=700808144	20	720	f
5048	3	save tbl=1015223380	20	720	f
5048	3	save tbl=1053058945	20	720	f
5048	3	save tbl=1053058945	20	720	f
5048	3	project	10	0	f
5048	4	return	10	259	f

Query 3 :-

```sql
with yellow_t as(
    select round(avg(trip_distance), 2) as average_trip_distance
    from spectrum_schema.yellow_taxi
    where passenger_count > 2
),
green_t as(
    select round(avg(trip_distance), 2) as average_trip_distance
    from spectrum_schema.green_taxi
    where passenger_count > 2
)

select 'yellow' as taxi_type,
       average_trip_distance
from yellow_t

union all

select 'green' as taxi_type,
       average_trip_distance
from green_t;
```

Execution Time :-

Query 5127

Completed, started on April 13, 2026 at 13:46:37

ELAPSED TIME: 00 m 12 s

Execution plan :-

Rows returned (35)

Export

Search rows

1 2 3 4

query	step	label	rows	bytes	is_diskbased
5127	0	scan tbl=214096123 name=S3 Subquery dev_spectrum_schema_yellow_taxi	6	96	f
5127	0	scan tbl=1785420196 name=Internal Worktable	2	48	f
5127	0	scan tbl=5614 name=Internal Worktable	1	28	f
5127	0	scan tbl=1534540520 name=Internal Worktable	2	56	f
5127	0	scan tbl=876706487 name=S3 Subquery dev_spectrum_schema_green_taxi	6	96	f
5127	0	scan tbl=5613 name=Internal Worktable	2	48	f
5127	0	scan tbl=1924271700 name=Internal Worktable	2	48	f
5127	0	scan tbl=5616 name=Internal Worktable	2	48	f
5127	0	scan tbl=5617 name=Internal Worktable	1	28	f
5127	0	scan tbl=5618 name=Internal Worktable	2	56	f

Data Scanned :-

query	mb_scanned	rows_scanned
5127	0	28

Data redistribution metrics :-

query	step	label	rows	bytes	is_diskbased
5127	0	scan tbl=2119519492 name=Internal Worktable	1	24	f
5127	0	scan tbl=5613 name=Internal Worktable	2	48	f
5127	0	scan tbl=1320182653 name=Internal Worktable	1	24	f
5127	0	scan tbl=5616 name=Internal Worktable	2	48	f
5127	0	scan tbl=214096123 name=S3 Subquery dev_spectrum_schema_yellow_taxi	6	96	f
5127	0	scan tbl=5618 name=Internal Worktable	2	56	f
5127	0	scan tbl=5614 name=Internal Worktable	1	28	f
5127	0	scan tbl=1785420196 name=Internal Worktable	2	48	f
5127	0	scan tbl=876706487 name=S3 Subquery dev_spectrum_schema_green_taxi	6	96	f
5127	0	scan tbl=1534540520 name=Internal Worktable	2	56	f
5127	0	scan tbl=1924271700 name=Internal Worktable	2	48	f
5127	0	scan tbl=5617 name=Internal Worktable	1	28	f
5127	1	project	6	0	f
5127	1	project	6	0	f
5127	1	project	2	0	f
5127	1	return	2	48	f
5127	1	return	2	35	f
5127	1	return	2	48	f
5127	1	aggr tbl=1320182653	1	24	f
5127	1	aggr tbl=2119519492	1	24	f
5127	1	project	1	0	f
5127	1	project	1	0	f
5127	2	return	2	56	f
5127	2	project	6	0	f
5127	2	project	1	0	f
5127	2	project	6	0	f
5127	2	save tbl=1534540520	1	28	f
5127	2	project	1	0	f

Task 6: Query Optimization

Next, load the data into Redshift with appropriate distribution and sort keys.
Re-run the above queries and compare the performance before and after applying these keys.

The intent of this task is to analyze how table design decisions affect query execution based on data volume and access patterns.

Query 1:-

```sql
with yellow_pdt as (
    select DATE(tpep_pickup_datetime) as pickup_date,
           payment_type,
           'yellow' as taxi_type,
           count(*) as total_trips_per_day
    from public.yellow_taxi_opt
    group by pickup_date, payment_type
    order by pickup_date, payment_type
),
green_pdt as (
    select DATE(lpep_pickup_datetime) as pickup_date,
           cast(payment_type as BIGINT),
           'green' as taxi_type,
           count(*) as total_trips_per_day
    from public.green_taxi_opt
    group by pickup_date, payment_type
    order by pickup_date, payment_type
)

select * from yellow_pdt
union all
select * from green_pdt;
```

Query 3289

Completed, started on April 13, 2026 at 18:11:33

ELAPSED TIME: 00 m 03 s

Query 2 :-

```sql
with yellow_t as (
    select 'yellow' as taxi_type,
           PULocationID as pickup_location_zone,
           DOLocationID as dropoff_location_zone,
           fare_amount
    from public.yellow_taxi_opt
),
green_t as (
    select 'green' as taxi_type,
           PULocationID as pickup_location_zone,
           DOLocationID as dropoff_location_zone,
           fare_amount
    from public.green_taxi_opt
),
combined as (
    select * from green_t
    union all
    select * from yellow_t
)

select * from combined
order by fare_amount desc
limit 10;
```

Query 3346

Execution | Data | Visualize

Completed, started on April 13, 2026 at 18:15:58
ELAPSED TIME: 00 m 02 s

Rows returned (10)

Export

Search rows

1

taxi_type	pickup_location_zone	dropoff_location_zone	fare_amount
yellow	239	132	187502.96
yellow	249	90	143163.45
yellow	43	264	19152.9
yellow	163	264	12015.47
yellow	48	125	6339
green	92	265	3523.3
yellow	233	40	2320.11
yellow	145	145	2194.7
yellow	132	265	1916.8
yellow	132	265	1726.6

Query 3 :-

```sql
with yellow_t as(
    select round(avg(trip_distance), 2) as average_trip_distance
    from public.yellow_taxi_opt
    where passenger_count > 2
),
green_t as(
    select round(avg(trip_distance), 2) as average_trip_distance
    from public.green_taxi_opt
    where passenger_count > 2
)

select 'yellow' as taxi_type,
       average_trip_distance
from yellow_t

union all

select 'green' as taxi_type,
       average_trip_distance
from green_t;
```

Query 3369

Completed, started on April 13, 2026 at 18:17:58

ELAPSED TIME: 00 m 07 s

Rows returned (2)

Search rows

taxi_type	average_trip_distance
yellow	3.9200000000000004
green	3.1399999999999997

Task 7: Performance Comparison and Analysis

Compare the results from before and after optimization and explain:

- Differences in execution time
- Changes in query execution plans
- Impact of data volume on optimization effectiveness
- Why certain queries benefited more than others

1. Differences in Execution Time

Before optimization:

- Queries executed on:
  - **Spectrum tables (S3)** OR
  - Tables without keys
- Result:
  - Full table scans
  - Higher latency
  - Slower execution

After optimization:

- Queries executed on:
  - **Internal Redshift tables with DISTKEY + SORTKEY**
- Result:
  - Reduced scan size
  - Faster aggregation
  - Lower execution time

Observation:

Execution time decreased significantly after applying distribution and sort keys due to reduced data movement and efficient data access.

2. Changes in Query Execution Plans

Before Optimization

- **DS_BCAST / DS_DIST_ALL**
  → Data redistributed across nodes
- Sequential scan on entire dataset
- No pruning of data

Meaning:

- High network cost
- High I/O

After Optimization

- **DS_DIST_KEY**
  → Data already co-located (no shuffle)
- **SORTKEY pruning**
  → Only relevant data scanned
- Reduced intermediate steps

Meaning:

- Less network transfer
- Efficient execution

Observation:

Query plans improved by reducing data redistribution and enabling efficient scanning using sort keys.

3. Impact of Data Volume on Optimization

Small Dataset

- Minimal performance difference
- Overhead dominates

Large Dataset (Your Case)

- Huge improvement because:
    - Less data scanned
    - Better parallelism
    - Reduced shuffle

Observation:

Optimization becomes more effective as data volume increases, as it minimizes unnecessary data movement and scanning.

4. Why Some Queries Benefited More

Query 1 (GROUP BY payment_type + date)

✓ Benefited MOST

- DISTKEY → optimized grouping
- SORTKEY → optimized date aggregation

Query 2 (ORDER BY fare_amount)

✓ Moderate improvement

- SORTKEY helps sorting
- But still requires ordering step

Query 3 (WHERE passenger_count > 2)

✓ Improved

- SORTKEY helps filtering
- Less data scanned